DREAM TEAM FLOW

A flow that learns, scales, and enforces — for Claude Code

The flow is the product, not the agents. Every session makes the next one better.

Demo Presentation | 30-60 min

Why DTF?

Claude Code is powerful out of the box. But running it on real tickets across a team exposes three gaps:

No memory between sessions

Claude makes the same mistakes twice. Your conventions live in your head, not in the system.

DTF:

6 learning paths feed improvements back automatically. Hooks capture → analyze → promote → skills, conventions, scripts, or memory.

No structure at scale

5 parallel tickets = 5 sessions that don't coordinate, don't share learnings, and burn tokens on deterministic work.

DTF:

Unlimited parallel tickets. Deterministic scripts for mechanical work (zero tokens). Early triage kills tickets before tokens are spent.

No guardrails you can trust

Claude can ignore a CLAUDE.md instruction. It can skip a step. "Best effort" isn't enough.

DTF:

Hooks enforce and can't be bypassed. Quality gates are deterministic. Human-in-the-loop at every gate. AI proposes, human disposes.

THE CORE

The flow is the product, not the agents. Agents are interchangeable — the orchestration, quality gates, and learning loops are what compound over time.

AGENDA

01	**Base Knowledge**	The 7 building blocks of Claude Code
02	**Self-Learning**	Your standards, your pitfalls, your codebase
03	**Parallel Everything**	Your review capacity is the bottleneck, not code
04	**Lean by Design**	Cut your AI spend, not your output
05	**Full Team vs Lite**	When to use which mode
06	**Built for Teams**	Onboard in minutes, not days
07	**Secure by Default**	Compliance without slowing down
08	**Marketplace**	Plugins for every role — dev, data, PO
10	**Best-in-Class Comparison**	Stripe · Claude Docs · ECC · DTF
10	**Live Demo**	Full ticket lifecycle + Q/A

01

Base Knowledge

The 7 building blocks of Claude Code instruction delivery

7 Ways to Instruct Claude Code

Each mechanism has one job — use the right one to avoid duplication and drift

Mechanism	Loading	Scope	Size	Best For
CLAUDE.md	Auto-loaded	Directory	Small	Project context, key paths, pointers
Skills	On-demand	Reusable	Large	Heavy conventions, tool guides, coding style
Commands	User /slash	Session	Large	Multi-step workflows, orchestration
Agents	Spawned	Task	Medium	Specialized roles, parallel work
Hooks	Automatic	Event	Tiny	Quality gates, enforcement, guardrails
Memory	Auto-loaded	Cross-session	Small	Learnings, preferences, patterns
Plugins	Installed	Community	Varies	Shared tools across teams/projects

CLAUDE.md / AGENTS.md

Auto-loaded directory context — always in the prompt, zero effort

USE FOR

Project structure & quick-start commands
Key file paths and references
Service-specific gotchas
Pointers: "see backend-conventions skill"

The sticky note on your monitor

DON'T USE FOR

Full coding conventions (too large!)
Step-by-step workflows (use commands)
Enforcement rules (use hooks)

Anti-pattern: pasting the entire style guide here — clutters every prompt

KEY INSIGHT

Every line in CLAUDE.md is loaded into every prompt when working in that directory.
Keep it small. Point to skills for heavy content. Think: table of contents, not encyclopedia.

MedHelp pattern: CLAUDE.md is a symlink to AGENTS.md — one file serves both Claude Code and other AI tools. Edit AGENTS.md, both consumers see it.

Skills

On-demand knowledge — loaded only when explicitly invoked, can be large

USE FOR

Heavy convention docs (full coding style)
Tool-specific guides (Playwright, Mermaid)
Reusable workflows (visual dev, code gen)
Knowledge that multiple agents need

The reference manual you pull off the shelf

DTF EXAMPLES

frontend-conventions (React/TS style guide)
backend-conventions (C#/.NET style guide)
data-conventions (EF Core, migrations)
playwright-cli (browser testing guide)
mermaid-diagram (diagramming syntax)
visual-development-workflow
strategic-compact (context management)
context-modes (dev/review/research)

SKILLS vs CLAUDE.md — THE KEY DIFFERENCE

CLAUDE.md: auto-loaded, always in context, keep small
Skills: explicitly invoked, loaded on demand, can be large

Agents READ skills when they need conventions:
backend-dev agent spawned -> reads backend-conventions skill -> has full C# style guide

Commands

User-invoked /slash-commands that orchestrate multi-step workflows

ORCHESTRATION

/create-stories — full lifecycle
/my-dream-team — --lite --interview
/workspace-launch — create worktree
/workspace-cleanup — tear down

REVIEW & TRIAGE

/review-pr — PR review (3 depths)
fast | --full | --deep | --deep --full
/ticket-scout — batch sprint triage
/ticket-refine — deep quality gate
/ticket-examples — code patterns

LEARNING & CONFIG

/retro-proposals — route learnings
/pr-insights — review patterns
/sync-config — push to GitHub
/evolve — promote usage patterns
/reviewers — PR reviewer mgmt

COMMANDS vs SKILLS

Commands = action: 'Do this workflow for me' (orchestrate)
Skills = knowledge: 'Here is what you need' (inform)

A command USES a skill: /my-dream-team spawns agent -> reads conventions

Agents

Specialized subprocesses — own context window, own role, own model tier

Amara — Architect

Model: Opus

Analyzes ticket, plans architecture, sizes team

Kenji / Ravi — Backend Dev

Model: Sonnet

.NET/C# — models, migrations, API endpoints

Ingrid / Elsa — Frontend Dev

Model: Sonnet

React/TS — components, RTK Query, Playwright

Mei — Data Engineer

Model: Sonnet

Complex queries, reports, data mapping

Maya — PR Reviewer

Model: Opus

Convention checks, security scan, categorized feedback
--deep: 4 parallel agents + validation pass
Only confirmed issues survive (false positives

Lena — Visual Verifier

Model: Sonnet

Playwright e2e tests + committed screenshots

WHY AGENTS MATTER

Own context window — heavy exploration doesn't pollute the main conversation

Run in parallel — backend + frontend implement simultaneously

Model per role — Opus for architecture/review (quality), Sonnet for implementation (speed + cost)

Crash-recoverable — agents write notes to disk, can resume from where they left off

Hooks

Event-driven enforcement — shell scripts that fire automatically and can't be bypassed

PostToolUse

Trigger: Every tool call | Action: Tool usage logging for analytics

Edit / Write

Trigger: File modifications | Action: Migration guard, lock file guard, lint reminder

Notification

Trigger: Agent needs attention | Action: macOS desktop notification

TeammateIdle

Trigger: Agent going idle | Action: Must write notes + journal first

TaskCompleted

Trigger: Agent marks done | Action: Must have code changes + type checks

PreCompact

Trigger: Before compression | Action: Auto-save checkpoint to disk

THE ENFORCEMENT LAYER

An agent can IGNORE a CLAUDE.md instruction. An agent can SKIP reading a skill.
An agent CANNOT bypass a hook. Hooks are deterministic and non-negotiable.

Skills DOCUMENT the rule | Hooks ENFORCE the rule | CLAUDE.md POINTS to both

PR Review — 3 Depth Levels

The only AI reviewer that filters its own false positives

FAST (default)

`/review-pr 1670`

GitHub API only
No local checkout
Diff per file (memory-safe)
Auto-skip lock files + generated
OWASP security checklist
MUST FIX / SUGGESTION /
QUESTION / PRAISE

WHEN:
Quick feedback, small changes

vs. VANILLA CLAUDE

Unstructured prose in terminal. No GitHub posting. No security scan. No filtering.

FULL (--full)

`/review-pr 1670 --full`

Everything in Fast, PLUS:
Local worktree checkout
dotnet build (backend)
tsc --noEmit (frontend)
eslint on changed files
vitest on changed tests
Full file context

WHEN:
Type errors invisible to API

DEEP (--deep)

`/review-pr 1670 --deep`

Everything in Fast, PLUS:
4 parallel review agents:
  2x Sonnet (conventions)
  2x Opus (bugs + security)
Every finding validated
Only CONFIRMED survives
Combine: --deep --full

WHEN:
Critical PRs, zero false pos.

Memory & Plugins

MEMORY — Cross-Session Persistence

Auto-loaded (first 200 lines of MEMORY.md)
Persists across conversations

4 types:
- user — your role, preferences, expertise
- feedback — corrections you've given Claude
- project — goals, deadlines, decisions
- reference — pointers to external systems

PLUGINS — Community Sharing

Install from marketplace repos
Add skills, commands, agents, hooks

```
/plugin marketplace add johwer/marketplace
/plugin install claude-toolkit@marketplace
```

Any repo can use any plugin
Repo-agnostic, team-shareable

THE COMPLETE PICTURE — Each Mechanism Has One Job

CLAUDE.md provides context Skills provide knowledge Commands orchestrate workflows

Agents do specialized work Hooks enforce rules Memory persists learnings

Plugins share it all across teams

When to Use What

1	Is it directory-specific context?	->	**CLAUDE.md**	(small, pointer-style)
2	Is it a full convention or reference doc?	->	**Skill**	(loaded on demand)
3	Is it a user-facing workflow?	->	**Command**	(/slash-command)
4	Is it a specialized role for parallel work?	->	**Agent**	(own context window)
5	Is it a rule that must be enforced?	->	**Hook**	(can't be ignored)
6	Is it a cross-session learning?	->	**Memory**	(persists forever)
7	Is it a shared community tool?	->	**Plugin**	(marketplace)

02

Self-Learning

Your standards, your pitfalls, your codebase

6 Learning Channels

Every ticket makes the system smarter — the same mistake never happens twice

Session Retros Agents reflect after every ticket	->	**Routes to:** Agent prompts, coding style docs
PR Review Mining Patterns from 200+ merged PRs	->	**Routes to:** Convention updates, review templates
Jira Pushback AI reviewer comment analysis	->	**Routes to:** Ticket triage rules, quality scoring
Research & Reading Articles, docs, best-practice guides	->	**Routes to:** Backend/frontend conventions
Tool Usage + Cross-Session Hooks capture patterns + cross-retro analysis	->	**Routes to:** Skills, conventions, scripts, process improvements

COMPOUNDING

Learnings feed into:

- Agent prompts
- Coding style docs
- Pre-hydrated context
- Triage rules
- Quality gates

New team members inherit everything on day one.

Knowledge is in the system, not in someone's head.

03

Parallel Everything

Your review capacity is the bottleneck, not code

How Parallel Works

CROSS-TICKET PARALLELISM

Run as many tickets as you can review

Each gets its own:

- Git worktree (isolated code)
- Docker containers (isolated services)
- Vite dev server (unique port 3100-3199)
- Agent team (own context windows)

Worktrees + npm i run DURING pre-hydration

WITHIN-TICKET PARALLELISM

Backend + frontend implement simultaneously

Shared API contract as the handoff point:

Amara writes contract

-> Kenji implements API endpoint

-> Ingrid implements UI + generates types

-> Both push to same branch

PAUSE & RESUME

Agents write notes to disk at every step

Crash? Respawn with full context from disk

Pause overnight? Resume next morning

Zero token cost between sessions

No lost work, ever

PORT ISOLATION

Main stack: localhost:3000 (Vite)
localhost:5001-5006 (APIs)

Worktree PLRS-1234: localhost:3123 (Vite)
localhost:6001-6006 (APIs)

All run simultaneously, no conflicts

04

Lean by Design

Cut your AI spend, not your output

Where Tokens Go — and Where They Don't

Most developers here use Pro plan — every token counts

ZERO TOKENS

Formatting (CSharpier, Prettier)
Linting (ESLint)
Type checking (tsc, dotnet build)
CI polling (shell scripts)
Port allocation
Worktree creation
Docker builds
All shell scripts, not LLM calls

SMART TOKENS

Opus for architecture + review
Sonnet for implementation
Skills loaded only when needed
Agents read only what they need
No MCP servers (no overhead)
Disk-based memory (not context)
Targeted file reads, not full scan

RIGHT MODE

Full Team: complex, multi-service (3-4 pt tickets)

Lite: Claude decides agent usage (1-2 pt tickets)

Local: no PR/push, experiments (zero overhead)

CONTEXT WINDOW AWARENESS (NEW)

Status line shows real-time context usage: [green] 42% ctx | Opus | $0.55
Color thresholds: [green] 0-69% | [yellow] 70-89% | [red] 90%+

Strategic compaction at phase boundaries, never mid-implementation

05

Full Team vs Lite

When to use which mode — and why

Three Modes

FULL TEAM

7-phase orchestration
Named agents (Amara, Kenji, Ingrid...)
Architecture-first approach
Parallel backend + frontend
Code review (Maya / --deep)
Visual verification (Lena)
Security scan (OWASP)
Draft PR from phase 1.5

WHEN:
3-4 point tickets
Multi-service changes
New features with UI + API
Domain model changes

/my-dream-team <ticket>

LITE

Claude decides agent usage
May spawn 0-2 agents
Explore → Plan → Implement → Commit
All quality gates still apply
Strategic compaction (auto)
Context modes + de-sloppify

WHEN:
1-2 point tickets
Single file changes
Bug fixes
Config/styling tweaks

/my-dream-team --lite <ticket>

LOCAL

No PR, no push
No Jira integration
Just code in the worktree
Quality gates still apply locally

WHEN:
Experiments, prototypes
Exploring an approach
Learning the codebase

/my-dream-team --local <ticket>

06

Built for Teams

Onboard in minutes, not days

One Command to Install

FOR NEW TEAM MEMBERS

dtf install <URL> --company-config config.json

What happens:

1. Interactive wizard (name, paths, terminal)
2. Symlinks all commands/scripts into ~/.claude/
3. De-sanitizes generic names -> real company names
4. Generates personal CLAUDE.md from template

5 minutes from zero to full setup

ONGOING MAINTENANCE

dtf update
  Pull latest, verify symlinks, regenerate CLAUDE.md

dtf doctor
  Health check: config, symlinks, required tools

dtf contribute
  Export retro learnings as PR to shared repo

MAC-FIRST, MULTI-TERMINAL

Designed for macOS (primary audience)
Supported terminals: Alacritty, iTerm2,
  Terminal, Kitty, WezTerm, Ghostty, Warp

Linux + WSL also supported

COMPANY CONFIG

Shared by team lead: Jira domain,
  ticket prefix, service names, paths

Personal config (never committed):
  Monorepo path, terminal, worktree dir

07

Secure by Default

Compliance without slowing down — high value for regulated industries

Built into the Pipeline, Not Bolted On

7-CATEGORY OWASP SCAN

Every PR scanned before human review:

1. SQL injection
2. Authentication / Authorization
3. Data exposure (PII, secrets)
4. XSS (dangerouslySetInnerHTML)
5. Path traversal
6. Insecure defaults (CORS, HTTPS)
7. Hardcoded secrets / credentials

THREE-TIER PERMISSION LADDER

Level 1: Personal sandbox
  Agent works in isolated worktree

Level 2: Shared standards
  Conventions enforced by hooks

Level 3: Team lockdown
  Schema changes require human approval

SCHEMA CHANGE GOVERNANCE

New migrations require:

- Mermaid ER diagram in PR description
- Explicit human approval before merge
- Seed data for local dev databases

Agents cannot autonomously change
your data model — they propose, you decide

QUALITY HOOKS (NON-NEGOTIABLE)

Migration guard: warns on migration edits
Lock file guard: prevents manual edits
Teammate idle gate: must write handoff notes
Task completed gate: must pass type checks

These are shell scripts — no agent can
bypass, ignore, or argue with a hook

08

Marketplace

Plugins for every role — dev, data science, PO

One Marketplace, Many Roles

DEVELOPERS

Full Dream Team workflow
PR review (fast/full/deep)
Worktree management
Quality gates & CI
Solomon AI bot integration

Current focus, most mature

17+ commands, 5 agents,
20+ scripts, 6 skills

DATA SCIENCE

Potential own agent setup:
- dbt model generation
- Statistics pipeline agents
- EF Core migration helpers
- Report generation workflows

Same plugin system,
different skills + agents

Marketplace is extensible

PRODUCT OWNERS

Potential tools:
- /ticket-scout for sprint planning
- /ticket-refine for quality gates
- Jira integration (ACLI)
- Requirement analysis

Non-technical users can benefit
from the same infrastructure

No code knowledge needed

THREE-REPO ARCHITECTURE

dream-team-flow (public)	DTF docs, dtf CLI, company config templates
marketplace (public)	Plugin marketplace — commands, agents, scripts (sanitized)
marketplace-private	Same files, unsanitized — for team use without setup

Team installs from private repo (real names). Community installs from public + company config.

09

Best-in-Class Comparison

How DTF combines the best of Stripe, Anthropic, and the community

Built on the Best — Not Reinvented

WHAT WE STOLE

STRIPE MINIONS

1,300+ PRs/week

Deterministic nodes
quality-gate.sh (zero tokens)

CI cap: 2 rounds max

Pre-flight context hydration
Early triage checkpoint

"Walls > model"
Hooks enforce + capture

CLAUDE DOCS

Anthropic's patterns

Verification = proof
Playwright e2e tests

Small CLAUDE.md
Skills carry the rest

Writer ≠ Reviewer
Fresh eyes review

Context anti-patterns
Strategic compaction

ECC (50K★)

Community toolkit

Continuous learning
hooks capture → patterns → skills

Context modes
Dev / Review / Research

De-sloppify pass
Phase 4.9, 8 categories

Config health scanner
config-scan.sh (A-F grade)

DTF ORIGINAL — What nobody else has

The complete flow: triage → hydrate → implement → verify → ship → learn

6 learning paths (most comprehensive of any system) · Deep PR review (4 agents + validation)

Human-in-the-loop at every gate · One-command team deployment · Unlimited parallel tickets

09

Live Demo

Full ticket lifecycle — from Jira to merged PR

For Product Owners

You don't need to code to get value from DTF

TICKET QUALITY

/ticket-scout

Batch triage before sprint

Story points 1-4 auto-estimated

Verdict: READY / REFINE / SKIP

/ticket-refine

Deep quality gate per ticket

Pushes back on missing ACs

Posts concrete Jira comment

Result:

Fewer 'what did you mean?' cycles

VISIBILITY

Draft PR from minute one

Follow progress on GitHub

See the diff grow in real time

How to Test section

Step-by-step verification

Exact URLs and test data

Anyone can verify without asking

Jira completion comment

PR link + summary auto-posted

Creator @mentioned when done

THROUGHPUT

Unlimited parallel tickets

Your bottleneck = review capacity

Not coding speed

Early triage checkpoint

You decide GO/SKIP before

any tokens are spent

Your context matters most

--interview flag

Claude asks YOU the hard

questions before coding starts

NO CODE KNOWLEDGE NEEDED

/ticket-scout and /ticket-refine work from Jira data only — no codebase access required. POs can run these independently.

Demo Flow

2 min	**DTF Setup**	Show ~/.claude/ structure, dtf doctor, settings.json
3 min	**Status Line**	Live context %, cost, model in terminal status bar
3 min	**Ticket Triage**	/ticket-scout -> /ticket-refine for one ticket
5 min	**Workspace**	/create-stories -> early triage (GO/SKIP) -> worktree + terminal
10 min	**Dream Team**	Amara analyzes -> Kenji + Ingrid implement in parallel
5 min	**Quality Gates**	quality-gate.sh, hooks fire, Maya reviews OR: /review-pr --deep (4 agents + validation)
3 min	**Ship It**	User confirms, PR ready, Jira updated, retro saved

Total: ~30 min demo + remaining time for questions, deep-dives, and exploration

10

Q/A & Discussion

Questions, deep-dives, what-ifs, next steps

DREAM TEAM FLOW

A flow that learns, scales, and enforces — for Claude Code

Get started:

```
dtf install https://github.com/johwer/dream-team-flow --company-config company-config.json
```

github.com/johwer/dream-team-flow

github.com/johwer/marketplace